

Tema 06: Asincronía y comunicación con el servidor

RA 07

Parte 01

1. Introducción

En las aplicaciones web modernas es habitual comunicarse con un servidor sin recargar la página y gestionar operaciones que no son inmediatas (peticiones HTTP, temporizadores, acceso a APIs, etc.).

JavaScript es **monohilo**. Solo puede ejecutar una tarea a la vez. Cada instrucción espera a que termine la anterior. Sin embargo, JavaScript si permite operaciones que tardan tiempo en segundo plano sin bloquear la aplicación:

- Peticiones HTTP
- Timers (setTimeout)
- Acceso a APIs
- Lectura de ficheros (en Node.js).

Esto se conoce como Asincronía.

```
console.log("Inicio");

setTimeout(
  () => {
    console.log("Esto llega después");
  },
  2000
);

console.log("Fin");
```

Salida:

```
Inicio
Fin
Esto llega después
```

Para resolver estos problemas aparecen dos conceptos clave en JavaScript:

- Promesas → Mecanismo del lenguaje para gestionar operaciones que tardan un tiempo (asíncronas).
- AJAX → Técnica para comunicar cliente-servidor de forma asíncrona.

Conceptos relacionados:

- fetch → Implementa AJAX y devuelve promesas
- async/await → Forma moderna de trabajar con promesas

Aunque suelen usarse juntas, **no son lo mismo**.

Una **Promesa** es un objeto que representa un valor que puede llegar ahora, puede llegar más tarde o puede fallar. Soporta 3 estados: pending, fulfilled o rejected. Se encarga de la gestión moderna de asincronía.

AJAX es una técnica de desarrollo web, no una tecnología concreta, que permite enviar y recibir datos del servidor sin recargar la página de forma asíncrona. Como sabes inicialmente se usaba XML como lenguaje de intercambio. Hoy el intercambio de datos se hace principalmente con JSON.

Ejemplos típicos con AJAX:

- Cargar datos de una API REST
- Enviar formularios sin recargar la página
- Actualizar partes concretas del DOM
- Autocompletados, búsquedas dinámicas, dashboards, ...

2. JavaScript Asíncrono (Asynchronous JavaScript)

https://www.w3schools.com/js/js_callback.asp

https://www.w3schools.com/js/js_asynchronous.asp

Como hemos visto, en JavaScript, las funciones que se ejecutan en paralelo con otras, sin bloquear la ejecución del programa, se llaman **funciones asíncronas**. Es decir, JavaScript permite delegar tareas y continuar ejecutando el resto del código.

Ya hemos visto en los temas anteriores que un callback es una función que se pasa como argumento a otra función y se ejecuta más tarde.

```
function myDisplayer(some) {
    document.getElementById("demo").innerHTML = some;
}

function myCalculator(num1, num2, myCallback) {
    let sum = num1 + num2;
    myCallback(sum);
}

myCalculator(5, 5, myDisplayer); // Recuerda: sin ()
```

Los ejemplos de callbacks vistos hasta ahora no son especialmente espectaculares y su objetivo era aprender la sintaxis y entender cómo se pasa una función como parámetro.

Es más, este ejemplo NO es asíncrono todavía, es solo un ejemplo para repasar la sintaxis.

Un ejemplo asíncrono sencillo y típico es:

```
function prepararSalto() {  
  document.getElementById("demo").innerHTML = "¡Salto al hiperespacio!";  
}  
  
console.log("Inicio misión");  
setTimeout(prepararSalto, 3000);  
  
console.log("Comprobando sistemas");
```

Recuerda que en lugar de pasar el nombre de una función, también se puede pasar una función anónima completa.

Aunque funcionan, los callbacks pueden generar código difícil de leer y depurar, así como lo que se conoce como: **Callback Hell**:

```
entrenarPadawan(() => {  
  construirSable(() => {  
    irAMision(() => {  
      volverAVivo();  
    });  
  });  
});
```

Esto no escala en proyectos reales y por este motivo, JavaScript propuso el uso de Promesas y Async / Await. Es decir, **los callbacks son la base conceptual, pero no la solución final**.

3. Promesas

https://www.w3schools.com/js/js_promise_reference.asp

En JavaScript moderno, las Promesas son el mecanismo estándar para gestionar operaciones asíncronas. Las promesas reemplazan a los callbacks para gestionar asincronía de forma limpia, escalable y profesional.

En programación asíncrona distinguimos dos tipos de código:

- a. **Producing code:** Es el código que tarda tiempo. Produce un resultado en el futuro

Ejemplos:

- Peticiones HTTP
- Temporizadores
- Lectura de archivos

- b. **Consuming code:** Es el código que necesita el resultado. Tiene que esperar. Se ejecuta cuando el resultado está listo.

Una Promesa es el objeto que conecta ambos mundos.

Una promesa puede estar en uno de estos tres estados:

- pending
- fulfilled
- rejected

Sintaxis general:

```
//Declaración:
let myPromise = new Promise(function (resolve, reject) {
  // Producing code (puede tardar)
  if (exito) {
    resolve(valor); // éxito
  } else {
    reject(error); // fallo
  }
});
```

```
//Uso clásico:
myPromise.then( // Consuming code
  function (valor) { /* si todo va bien */ },
  function (error) { /* si algo falla */ }
);
```

```
//Uso más moderno:
myPromise
  .then(function (valor) {
    // consume éxito
  })
  .catch(function (error) {
    // consume error
  });
```

Nota: reject y error son opcionales.

Una vez que una promesa se resuelve o se rechaza, su estado no cambia.

Vemos un ejemplo:

```
//Declaración:
let promesaMision = new Promise(function (resolve, reject) {
  let exito = true;

  // Código que puede tardar tiempo
  if (exitito) {
    resolve("Misión completada");
  } else {
    reject("Misión fallida");
  }
});
```

```
//Uso clásico:
promesaMision.then(
  function (resultado) { console.log(resultado); },
  function (error) { console.error(error); }
);
```

```
// Uso más moderno:
promesaMision
  .then(function (resultado) { console.log(resultado); })
  .catch(function (error) { console.error(error); });
```

Nota: Como es lógico también puedes usar funciones flecha

```
promesaMision
  .then ( resultado => console.log(resultado) )
  .catch( error => console.error(error) );
```

Aunque conceptualmente una promesa tiene state y result **NO** se pueden consultar directamente.

Otro ejemplo:

Promesa	Callback
<pre>let promesa = new Promise(function (resolve) { setTimeout(function () { resolve("La Fuerza te acompaña"); }, 3000); }); promesa.then(valor => { document.getElementById("demo").inne rHTML = valor; });</pre>	<pre>setTimeout(function () { mostrarMensaje("La Fuerza te acompaña"); }, 3000); function mostrarMensaje(texto) { document.getElementById("demo").inne rHTML = texto; }</pre>
Una promesa dice "habrá un resultado en el futuro". - El código asíncrono queda desacoplado. - Manejo de errores centralizado	Un callback dice "qué hacer después". - Es difícil encadenar más funciones. - No hay forma estándar de manejar errores.

Uso de catch y finally:

```
promesaMision
  .then( resultado => texto = resultado )
  .catch( error => texto = error )
  .finally(
    () => {
      document.getElementById("demo").innerHTML = texto;
    }
  );
```

Uso de Promise.allSettled:

Promise.allSettled() ejecuta varias promesas a la vez y espera a que todas terminen, sin importar si fallan o no. Uso típico: logs, estadísticas o monitorización. Es decir, cuando no importa el resultado, solo que terminen.

```
const mision1 = new Promise(resolve => setTimeout(resolve, 200, "Jedi"));
const mision2 = new Promise(resolve => setTimeout(resolve, 100, "Sith"));

Promise.allSettled([mision1, mision2])
  .then(resultados => {
    resultados.forEach(r => console.log(r.status));
  });
```

3.1. Async / await

https://www.w3schools.com/js/js_async.asp

Async y await son la forma moderna y recomendada de trabajar con promesas en JavaScript. **No eliminan la asincronía, solo hacen el código más legible.**

- async → hace que una función devuelva siempre una Promesa
- await → hace que la función espere a que una Promesa se resuelva.

⚠ Await solo puede usarse dentro de una función async

Ejemplo básico:

```
async function mostrarMensaje() {  
  
    // 1. Creamos la promesa (simula una operación)  
    let promesaMensaje = new Promise(function (resolve) {  
        // Se espera a que termine una operación  
        resolve("La Fuerza te acompaña");  
    });  
  
    // 2. Esperamos la promesa (consuming code)  
    const resultado = await promesaMensaje;  
  
    // 3. Usamos el resultado  
    document.getElementById("demo").innerHTML = resultado;  
}  
  
mostrarMensaje();
```

Ejemplo con un posible error en la promesa

```
async function prepararSalto() {  
  
    // 1. Creamos la promesa (simula una tarea que puede tardar)  
    let promesaSalto = new Promise(function (resolve, reject) {  
  
        // Aquí se esperaría a que termine una operación larga  
        let energia = false; //simulamos el resultado de esa operación  
  
        if (energia) {  
            resolve(";Salto al hiperespacio realizado!");  
        }  
        else {
```

```
        reject("Error: no hay energía suficiente");
    }
});

// 2. Esperamos el resultado de la promesa
try {
    let resultado = await promesaSalto;
    document.getElementById("demo").innerHTML = resultado;
} catch (error) {
    document.getElementById("demo").innerHTML = error;
}

prepararSalto();
```

Flujo:

- Se crea la promesa
- **await** espera su resultado
- Se continúa cuando está resuelta

4. AJAX

AJAX significa Asynchronous JavaScript And XML (aunque hoy se usa habitualmente JSON o texto plano).

Es una técnica de desarrollo web. AJAX se basa en la combinación de:

- Un objeto del navegador para hacer peticiones HTTP
 - XMLHttpRequest (clásico) // fetch (moderno)
- JavaScript asíncrono
- HTML DOM, para mostrar los datos recibidos

Cómo funciona AJAX (todo ocurre sin recargar la página):

1. Ocurre un evento en la página (se carga la página o se pulsa un botón)
2. JavaScript crea un objeto de petición
3. Se envía una petición HTTP al servidor
4. El servidor procesa la petición
5. El servidor responde (JSON, texto, etc.)
6. JavaScript lee la respuesta
7. JavaScript actualiza el DOM

4.1. AJAX CLÁSICO

https://www.w3schools.com/js/js_ajax_http.asp

https://www.w3schools.com/js/js_ajax_http_send.asp

https://www.w3schools.com/js/js_ajax_http_response.asp

El corazón de AJAX clásico es el objeto **XMLHttpRequest**.

Siempre se siguen estos 4 pasos para hacer la petición:

- Crear el objeto XMLHttpRequest
- Definir un callback (qué hacer cuando llega la respuesta)
- Abrir la petición (open)
- Enviar la petición (send)

```
const xhttp = new XMLHttpRequest();

xhttp.onload = function () {
    // Usar la respuesta
};

xhttp.open("GET", "archivo.txt");
xhttp.send();
```

El callback que espera la respuesta puede hacerse de dos formas

a. Callback: onload - Forma recomendada

```
xhttp.onload = function () {
    if (this.status === 200) {
        console.log(this.responseText);
    }
};
```

b. Callback: onreadystatechange - Forma antigua. Por curiosidad

```
xhttp.onreadystatechange = function () {
    if (this.readyState === 4 && this.status === 200) {
        console.log(this.responseText);
    }
};
```

Al abrir la petición puede hacerse con GET o POST:

a. GET - Más simple

```
xhttp.open("GET", "datos.php?planeta=Tatooine");
xhttp.send();
```

b. POST - Más seguro

```
xhttp.open("POST", "datos.php");
xhttp.setRequestHeader("Content-type",
    "application/x-www-form-urlencoded");
xhttp.send("planeta=Tatooine");
```

Por defecto, todas las peticiones son asíncronas.

Si quisieras que la petición fuera síncrona (nada recomendado):

```
xhttp.open("GET", "archivo.txt", false);
```

Vemos un ejemplo completo usando una API externa que devuelve un JSON y AJAX clásico:

<https://swapi.dev> | <https://swapi.info/>

```
//Ejemplo del JSON devuelto:
{
  "name": "Luke Skywalker",
  "height": "172",
  "mass": "77",
  "hair_color": "blond",
  "skin_color": "fair",
  "eye_color": "blue",
  "birth_year": "19BBY",
  "gender": "male",
  "homeworld": "https://swapi.dev/api/planets/1/",
  "films": [
    "https://swapi.dev/api/films/2/",
    "https://swapi.dev/api/films/6/",
    "https://swapi.dev/api/films/3/",
    "https://swapi.dev/api/films/1/",
    "https://swapi.dev/api/films/7/"
  ],
  "species": [
    "https://swapi.dev/api/species/1/"
  ],
  "vehicles": [
    "https://swapi.dev/api/vehicles/14/",
    "https://swapi.dev/api/vehicles/30/"
  ],
  "starships": [
    "https://swapi.dev/api/starships/12/",
    "https://swapi.dev/api/starships/22/"
  ],
  "created": "2014-12-09T13:50:51.644000Z",
  "edited": "2014-12-20T21:17:56.891000Z",
  "url": "https://swapi.dev/api/people/1/"
}
```

```
function cargarJedi() {
  const xhttp = new XMLHttpRequest();

  // Definir el callback
  xhttp.onload = function () {
    if (this.status === 200) {
      let jedi = JSON.parse(this.responseText);

      document.getElementById("resultado").innerHTML =
        "Nombre: " + jedi.name + "<br>" +
        "Altura " + jedi.height + "<br>" +
        "Género: " + jedi.gender;
    }
    else {
      console.error("Error en la petición");
    }
  };

  xhttp.open("GET", "https://swapi.dev/api/people/1/", true);

  xhttp.send();
}
```

Si quisiéramos mandar datos con POST y XMLHttpRequest

```
function crearJedi() {
  const xhttp = new XMLHttpRequest();

  xhttp.onload = function () {
    if (this.status === 200 ) {
      console.log("Jedi creado correctamente");
    } else {
      console.error("Error al crear el Jedi");
    }
  };

  xhttp.open("POST", "https://example.com/api/jedi", true);
  xhttp.setRequestHeader("Content-Type", "application/json");

  const jedi = {
    nombre: "Luke Skywalker",
    planeta: "Tatooine",
    arma: "Sable láser"
  };

  xhttp.send(JSON.stringify(jedi));
}
```

4.2. AJAX MODERNO SIN ASYNC / AWAIT

https://www.w3schools.com/js/js_api_fetch.asp

En JavaScript moderno, el uso de AJAX se implementa principalmente mediante la Fetch API, que sustituye al antiguo objeto XMLHttpRequest.

La función fetch permite realizar peticiones HTTP de forma sencilla y clara, y su característica principal es que devuelve una Promesa, lo que la integra directamente con la programación asíncrona actual.

Para trabajar con esa promesa, se pueden utilizar métodos como .then() y .catch(), o bien la sintaxis más moderna y recomendable async / await, que hace que el código asíncrono sea más legible y parecido al código secuencial.

Vemos un ejemplo completo usando la API externa anterior:

```
function cargarJedi() {  
  fetch ("https://swapi.dev/api/people/1/")  
    .then(function (response) {  
      // response NO son los datos todavía  
      // Es la respuesta HTTP  
      return response.json(); // convierte JSON → objeto JS  
    })  
    .then(function (data) {  
      // data ya es un objeto JavaScript  
      document.getElementById("resultado").innerHTML =  
        "Nombre: " + data.name + "<br>" +  
        "Altura: " + data.height + "<br>" +  
        "Planeta: " + data.homeworld;  
    })  
    .catch(function (error) {  
      console.error("Error en la petición:", error);  
    });  
}
```

Si quisiéramos mandar datos con POST y fetch sin async / await

```
function crearJedi() {  
  fetch("https://example.com/api/jedi", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({  
      nombre: "Luke Skywalker",  
      planeta: "Tatooine",  
    })  
  })  
}
```

```
        arma: "Sable láser"
    })
})
.then(function (response) {
    return response.json();
})
.then(function (data) {
    console.log("Jedi creado:", data);
})
.catch(function (error) {
    console.error("Error en la petición:", error);
});
}
```

4.3. AJAX MODERNO CON ASYNC / AWAIT - Forma recomendada

https://www.w3schools.com/js/js_api_fetch.asp

Finalmente vemos el mismo ejemplo con async / await

Esta la forma recomendada porque es más legible, más cercano al código síncrono y más fácil de depurar:

```
async function cargarJedi() {
    try {
        const response = await fetch("https://swapi.dev/api/people/1/");
        const data = await response.json();

        document.getElementById("resultado").innerHTML =
            "Nombre: " + data.name + "<br>" +
            "Altura: " + data.height + "<br>" +
            "Planeta: " + data.homeworld;

    }
    catch (error) {
        console.error("Error en la petición:", error);
    }
}
```

Si quisiéramos mandar datos con POST y fetch con async / await

```
async function crearJedi() {
    try {
        const response = await fetch("https://example.com/api/jedi",
```

```
{
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    nombre: "Luke Skywalker",
    planeta: "Tatooine",
    arma: "Sable láser"
  })
});

const data = await response.json();
console.log("Jedi creado:", data);

} catch (error) {
  console.error("Error en la petición:", error);
}
}
```

5. JSON

https://www.w3schools.com/js/js_json.asp

Como sabes JSON (JavaScript Object Notation) es un formato de texto utilizado para intercambiar y almacenar datos, especialmente entre cliente y servidor.

```
{
  "name": "Luke Skywalker",
  "height": 172,
  "gender": "male",
  "jedi": true,
  "master": null
}
```

Vamos a repasar algunos aspectos importantes:

- Las claves van siempre entre **comillas dobles**
- Los valores pueden ser:
 - string
 - number
 - boolean
 - object
 - array
 - null

NOTA: NO admite undefined, function ni Date.

- Funciones nativas:

- a. **JSON.parse()**. Convierte texto JSON → objeto JavaScript

```
const texto = '{"name":"Darth Vader","side":"dark"}';
const personaje = JSON.parse(texto);
console.log(personaje.name);
```

- b. **JSON.stringify()**. Convierte objeto JavaScript → texto JSON

```
const jedi = {
  name: "Obi-Wan Kenobi",
  side: "light"
};

const json = JSON.stringify(jedi);
```

Recuerda: Esto es imprescindible para enviar datos con POST y guardar objetos en localStorage.

Cuando haces una petición AJAX, el servidor siempre envía texto y ese texto suele ser JSON

```
...
const response = await fetch("https://swapi.dev/api/people/1/");
const data = await response.json();
                //hace internamente un JSON.parse().
...
```

- Objetos JSON y Arrays JSON

```
{
  "name": "Yoda",
  "species": "Unknown"
}

[
  "Luke",
  "Leia",
  "Han"
]

{
  "characters": ["Luke", "Leia", "Han"]
}
// Ambos formatos son JSON válidos.
```

```
// La diferencia está en cómo se accede a los datos desde JS.  
  
{  
  "name": "Anakin Skywalker",  
  "films": [  
    "A New Hope",  
    "The Empire Strikes Back"  
  ]  
}
```

Anexo: Web APIs

https://www.w3schools.com/js/js_api_intro.asp

Una Web API es un conjunto de funcionalidades que el navegador pone a disposición de JavaScript para realizar operaciones complejas sin necesidad de programarlas desde cero.

Gracias a las Web APIs, JavaScript puede:

- Manipular la página web
- Acceder a datos del usuario
- Comunicarse con servidores
- Almacenar información
- Usar hardware del dispositivo (ubicación, sensores, etc.)

No son librerías externas: vienen integradas en el navegador.

A.1. Web APIs que ya hemos utilizado en el curso

A lo largo del curso ya hemos trabajado con varias Web APIs:

- DOM API: Para acceder y modificar elementos HTML (`document.getElementById`, `querySelector`, etc.).
- History API: Permite interactuar con el historial de navegación del navegador.
- Form Validation API: Para validar formularios desde HTML y JavaScript (`required`, `pattern`, `checkValidity`, etc.).
- Fetch API: Para realizar peticiones HTTP asíncronas a APIs externas (AJAX moderno).

A.2. Geolocation API

https://www.w3schools.com/js/js_api_geolocation.asp

Permite obtener la ubicación geográfica del usuario, siempre con su consentimiento. Devuelve latitud y longitud, pero **solo funciona en HTTPS**.

Puede fallar si el usuario rechaza el permiso, si el dispositivo no puede determinar la ubicación o si la conexión es imprecisa.

```
function obtenerUbicacion() {
  const salida = document.getElementById("ubicacion");

  if (!navigator.geolocation) {
    salida.textContent = "La geolocalización no está disponible.";
    return;
  }

  navigator.geolocation.getCurrentPosition(
    function (posicion) {
      salida.textContent =
        "Latitud: " + posicion.coords.latitude +
        " | Longitud: " + posicion.coords.longitude;
    },
    function () {
      salida.textContent = "El usuario no permite la geolocalización.";
    }
  );
}
```

Muy importante: La Geolocation API no muestra mapas por sí misma. Su función es únicamente obtener coordenadas (latitud y longitud). Para mostrar un mapa es necesario usar un servicio externo de mapas, que reciba esas coordenadas y las represente visualmente. La geolocalización se combina, por tanto, con una API de mapas.

Formas habituales de mostrar mapas:

- a. Imagen estática de mapa (la más simple). Se utilizan las coordenadas para generar una imagen de mapa. No es interactivo, pero es muy sencillo. Puede hacerse con:
 - Google Maps Static API (requiere clave)
 - OpenStreetMap (mediante servicios externos)

```

```

- b. Mapa interactivo

- Con Leaflet + OpenStreetMap (recomendado). Se usa una librería ligera como Leaflet junto con OpenStreetMap. No requiere clave API y es gratuita.

- Con Google Maps JavaScript API. Permite mostrar mapas muy completos y personalizables. Aunque requiere cuenta y clave API.

A.3. Web Storage API

https://www.w3schools.com/js/js_api_web_storage.asp

Las siguientes APIs forman la base moderna para almacenar datos en el navegador:

- a. localStorage → datos persistentes
- b. sessionStorage → datos temporales (se borran al cerrar el navegador)

IndexedDB API

Es una base de datos asíncrona y transaccional en el navegador que permite grandes cantidades de datos. Es más potente que localStorage y más compleja de usar.

Uso típico:

- Aplicaciones web avanzadas
- Apps offline
- PWAs

A.4. Web Worker API

https://www.w3schools.com/js/js_api_web_workers.asp

Permite ejecutar JavaScript en segundo plano, sin bloquear la interfaz.

Uso típico:

- Tareas pesadas
- Cálculos complejos
- Procesamiento de datos

Se menciona solo a nivel conceptual.

A.5. APIs externas

Además de las Web APIs del navegador, existen APIs externas que no vienen integradas:

- APIs REST públicas (Harry Potter, Pokémon, etc.)
- APIs de servicios externos (YouTube, Twitter, etc.)

Estas APIs **NO** forman parte del navegador

Muy importante: Seguridad y Web APIs

El uso de Web APIs implica responsabilidad en seguridad. Riesgos principales:

- XSS (Cross-Site Scripting)
- Uso incorrecto de innerHTML
- Almacenamiento de datos sensibles en el cliente

Buenas prácticas:

- No guardar contraseñas ni tokens sensibles en localStorage
- Preferir sessionStorage para datos temporales
- Validar y sanitizar cualquier dato antes de mostrarlo
- Evitar innerHTML cuando no sea necesario

Referencias Bibliográficas

Estos apuntes han sido elaborados con base en distintas fuentes:

- Apuntes del profesor José Castillo © Copyright 2023.
- MDN Web Docs (Mozilla Developer Network, 2023): documentación oficial sobre tecnologías web cliente/servidor, JavaScript y compatibilidad entre navegadores.
- OpenAI (2025): asistencia mediante ChatGPT-5, usado como apoyo en la redacción y organización de contenidos.